
Aplicaciones reales de los esquemas de compromiso

Sistemas desplegados en producción,
mecanismos criptográficos y referencias

Nueve dominios de aplicación —dinero confidencial, transparencia de certificados, votación verificable, custodia umbral, credenciales anónimas, subastas, aleatoriedad pública, almacenamiento verificable y aprendizaje automático auditable— con el esquema concreto que emplea cada sistema, su escala de despliegue y la literatura correspondiente.

Dr. José de Jesús Ángel Ángel

Lab-IA — Facultad de Ingeniería
Universidad Anáhuac México

20 de agosto de 2026

Índice general

1 El patrón común

Los esquemas de compromiso rara vez son el producto final: son la pieza que hace posible una propiedad que, sin ellos, sería contradictoria. En casi todos los sistemas de este documento el compromiso resuelve una de tres tensiones.

Las tres tensiones que resuelve un compromiso

1. **Ocultar y a la vez verificar.** Se necesita comprobar una propiedad de un dato sin conocerlo. Ejemplo: verificar que las entradas de una transacción igualan sus salidas sin ver ningún monto (Monero, Mimblewimble). Requiere *homomorfía*.
2. **Fijar antes de revelar.** Varias partes deben actuar sin conocer la acción de las demás, pero sin poder cambiar la propia después. Ejemplo: subastas de sobre cerrado, lanzamiento de moneda distribuido, aleatoriedad pública. Requiere *vinculación*.
3. **Publicar sin exponer y auditar después.** Una autoridad publica un resumen compacto de un registro enorme; cualquiera puede exigir después la prueba de que un elemento estaba incluido. Ejemplo: Certificate Transparency, transparencia de claves, sellado de tiempo. Requiere *sucintitud* y apertura parcial.

Observación 1.1. Una regla práctica útil al leer arquitecturas de sistemas: si el diseño exige *operar* sobre valores ocultos, casi seguramente hay un compromiso de Pedersen detrás. Si solo exige *demostrar inclusión* en un conjunto grande, hay un árbol de Merkle. Si exige ambas cosas a escala, hay un compromiso polinomial (KZG o FRI).

1.1 Cronología de despliegues

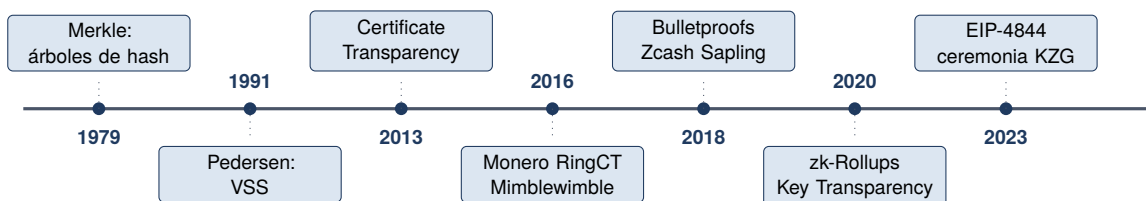


Figura 1: Del resultado teórico al despliegue masivo. Nótese la brecha de veinte años entre Pedersen (1991) y su primer uso a escala en dinero digital (2016).

2 Dinero confidencial

2.1 Transacciones confidenciales: el mecanismo base

Referencias: Maxwell (2015) [?]; Poelstra *et al.* (2018) [?].

La idea, propuesta por Maxwell para Bitcoin y desplegada primero en la sidechain Liquid de Blockstream, sustituye cada monto en claro por un compromiso de Pedersen $C = g^v h^r$. La validez de una transacción se reduce a una identidad de grupo:

$$\sum_{\text{salidas}} C_j - \sum_{\text{entradas}} C_i = h^{\Delta r},$$

es decir, la diferencia debe ser un compromiso a cero. El verificador confirma la conservación del valor sin conocer un solo monto.

Por qué la prueba de rango no es opcional

La aritmética ocurre módulo q . Sin restricción adicional, un usuario podría comprometerse a un monto «negativo» —un v cercano a q — y crear dinero de la nada conservando la ecuación. Cada salida requiere por tanto una **prueba de rango** que certifique $v \in [0, 2^{64})$.

Este es el componente caro: antes de Bulletproofs consumía varios kilobytes por salida. La reducción a ~675 bytes fue lo que hizo económicamente viables las transacciones confidenciales a escala [?].

2.2 Monero (RingCT)

Referencia: Noether *et al.* (2016) [?].

Monero activó RingCT en enero de 2017 y lo hizo obligatorio en septiembre del mismo año. Combina tres primitivas: compromisos de Pedersen para los montos, firmas de anillo para ocultar el emisor, y direcciones sigilosas para el receptor. En 2018 migró sus pruebas de rango a Bulletproofs, con una reducción reportada de aproximadamente 80 % en el tamaño de transacción y una caída proporcional en las comisiones.

El generador H se deriva por `hash_to_point` sobre la codificación del generador estándar de Ed25519, precisamente para garantizar que $\log_G H$ sea desconocido.

2.3 Mimblewimble: Grin y Beam

Referencias: Jedusor (2016) [?]; Poelstra (2016) [?].

Mimblewimble lleva la homomorfía a su conclusión estructural. Como toda salida es un compromiso y las sumas se cancelan, las transacciones intermedias pueden eliminarse de la cadena: si una salida fue creada y luego gastada, ambos compromisos se anulan mutuamente y desaparecen del histórico. Solo sobreviven el conjunto de salidas no gastadas y los «kernels» —firmas bajo la clave $h^{\Delta r}$ — que acreditan que cada transacción conservó el valor.

El resultado es una blockchain cuyo tamaño crece con el estado actual, no con la historia completa. Grin y Beam lanzaron en enero de 2019 como implementaciones independientes del mismo diseño.

Un caso donde la propiedad algebraica define la arquitectura

Mimblewimble no es «Bitcoin con privacidad añadida»: es un diseño que solo existe porque el compromiso de Pedersen es homomórfico. La poda de la cadena, la ausencia de direcciones y la agregación de transacciones (*CoinJoin* implícito) son todas consecuencias de la misma identidad algebraica. Es probablemente el mejor ejemplo de una primitiva criptográfica que determina la arquitectura completa de un sistema.

2.4 Zcash

Referencias: Ben-Sasson *et al.* (2014) [?]; Hopwood *et al.* [?].

Zcash usa compromisos de manera distinta: cada «nota» (una unidad de valor con su receptor) se representa por un compromiso publicado en un árbol de Merkle incremental. Gastar una nota consiste en demostrar en conocimiento cero que se conoce la apertura de *alguna* hoja del árbol, sin revelar cuál, junto con un *nullifier* determinista que impide el doble gasto.

La elección del esquema de compromiso ha evolucionado según las restricciones del circuito:

Versión	Esquema	Motivo
Sprout (2016)	SHA-256	Simplicidad; costoso en el circuito
Sapling (2018)	Hash de Pedersen (Jubjub)	Mucho más barato en restricciones R1CS
Orchard (2022)	Sinsemilla	Optimizado para el sistema Halo 2 sin setup

El hash de Pedersen usado en Sapling no es el compromiso de Pedersen estándar, sino una función de compresión construida sobre la misma estructura de suma de puntos: es resistente a colisiones bajo DLog y —lo decisivo— se evalúa dentro de un circuito aritmético con un orden de magnitud menos de restricciones que SHA-256.

3 Escalabilidad y disponibilidad de datos

3.1 EIP-4844: compromisos KZG en producción

Referencia: Buterin *et al.* (2022) [?].

La actualización *proto-danksharding* de Ethereum, activada en marzo de 2024, es probablemente el despliegue de compromisos polinomiales más grande hasta la fecha. Cada transacción puede transportar «blobs» de ~ 128 KB que no se almacenan en el estado; el bloque guarda únicamente un compromiso KZG de 48 bytes al polinomio que interpola el blob sobre el campo escalar de BLS12-381.

La ceremonia de configuración confiable más grande jamás realizada

KZG requiere una cadena de referencia estructurada $\{g^{\tau^i}\}$ donde τ debe ser desconocido. La *KZG Ceremony* de Ethereum (2023) reunió más de 140 000 contribuciones individuales en un cómputo multiparte secuencial.

La propiedad de seguridad es notablemente favorable: basta que **una sola** persona haya destruido honestamente su aportación para que τ permanezca desconocido. La confianza requerida no es en la mayoría, sino en la existencia de al menos un participante honesto —un umbral 1-de- n con n del orden de 10^5 .

3.2 Muestreo de disponibilidad de datos

Referencia: Al-Bassam *et al.* (2021) [?].

El problema: un nodo ligero necesita convencerse de que los datos de un bloque fueron efectivamente publicados, sin descargarlos. La solución combina codificación Reed–Solomon con compromisos: se compromete la extensión codificada y el nodo solicita muestras aleatorias con sus pruebas de apertura. Si el productor ocultó más de la mitad de los datos, unas pocas decenas de muestras lo detectan con probabilidad abrumadora. Es la base de Celestia y del diseño *danksharding* completo.

3.3 Árboles Verkle y clientes sin estado

Referencias: Kuszmaul (2019) [?]; Buterin (2021) [?].

El árbol de Merkle-Patricia de Ethereum produce testigos de estado de varios megabytes, lo que impide ejecutar clientes sin estado. Los árboles Verkle sustituyen los nodos hash por compromisos vectoriales KZG, permitiendo factores de ramificación de 256 sin que la prueba crezca con el ancho del nodo. La reducción proyectada es de megabytes a decenas de kilobytes.

3.4 Rollups de conocimiento cero

Los *zk-rollups* ejecutan transacciones fuera de la cadena y publican una prueba sucinta de la corrección del nuevo estado. Todos descansan sobre compromisos polinomiales, con una división clara según la tolerancia al setup de confianza:

Sistema	Compromiso	Compromiso de diseño
zkSync Era	KZG (PLONK)	Pruebas pequeñas; requiere ceremonia
Scroll	KZG (halo2-KZG)	Compatibilidad EVM a nivel de bytecode
Polygon zkEVM	KZG	Verificación barata en L1
StarkNet	FRI	Transparente y post-cuántico; pruebas mayores
Aztec	KZG / IPA	Privacidad además de escalabilidad

4 Transparencia e infraestructura de confianza

4.1 Certificate Transparency

Referencias: Laurie *et al.* (2013) [?]; Laurie *et al.* (2021) [?].

Es, en volumen, la aplicación de compromisos más masiva de Internet. Cada certificado TLS emitido por una autoridad pública debe registrarse en varios registros solo-anexables (*append-only logs*) estructurados como árboles de Merkle. El registro devuelve un SCT (*Signed Certificate Timestamp*) que el navegador exige.

Dos tipos de prueba sostienen el sistema:

- **Prueba de inclusión:** demuestra que un certificado concreto está en el registro con raíz r . Tamaño $O(\log n)$.
- **Prueba de consistencia:** demuestra que el árbol con raíz r_2 es una extensión del árbol con raíz r_1 , sin eliminaciones ni reescrituras. Es esta prueba la que convierte el árbol en un registro verdaderamente solo-anexable.

Chrome exige CT para todos los certificados desde abril de 2018. El sistema detectó en la práctica emisiones indebidas —el caso más citado es el de certificados de Symantec en 2015— que de otro modo habrían pasado inadvertidas.

4.2 Transparencia de claves en mensajería

Referencias: Melara *et al.* (2015) [?]; Chase *et al.* (2019) [?]; Malvai *et al.* (2023) [?].

El cifrado de extremo a extremo depende de obtener la clave pública correcta del interlocutor. Si el servidor puede sustituirla silenciosamente, el cifrado no protege de nada. La transparencia de claves resuelve esto publicando un compromiso al directorio completo de claves, de modo que cualquier sustitución quede registrada de forma auditable.

CONIKS [?] introdujo el diseño; SEEMless [?] añadió privacidad —el registro no revela qué usuarios existen— y Parakeet [?] lo llevó a escala de producción. WhatsApp desplegó transparencia de claves en 2023; Apple introdujo *iMessage Contact Key Verification* el mismo año.

4.3 Firma y procedencia de software

Referencia: Newman *et al.* (2022) [?].

Sigstore registra cada firma de artefacto de software en Rekor, un registro transparente de Merkle. La motivación son los ataques a la cadena de suministro —SolarWinds (2020), *event-stream*, *xz-utils* (2024)— donde un artefacto malicioso se distribuye con firma aparentemente válida. El registro no impide la firma maliciosa, pero garantiza que quede permanentemente visible.

4.4 Sellado de tiempo y preinscripción

OpenTimestamps agrega millones de hashes en un árbol de Merkle y ancla únicamente la raíz en una transacción de Bitcoin, distribuyendo el costo entre todos los documentos sellados. La aplicación no monetaria más interesante es la **preinscripción de hipótesis científicas**: un investigador publica $H(\text{protocolo} \| r)$ antes de recolectar datos, y lo revela al publicar. Esto impide el *HARKing*—formular la hipótesis después de conocer los resultados— con una garantía criptográfica en lugar de institucional.

5 Votación electrónica verificable

La votación impone requisitos que parecen incompatibles: el voto debe ser secreto, el recuento verificable por cualquiera, y el votante debe poder comprobar que su voto se contó —pero sin poder demostrar a un tercero por quién votó, para impedir la compra de votos. Los compromisos son la herramienta central de este equilibrio.

5.1 Helios

Referencia: Adida (2008) [?].

Cada voto se publica como un cifrado ElGamal exponencial, que funciona como compromiso perfectamente vinculante y computacionalmente ocultante. El recuento se hace multiplicando los cifrados —homomorfía— y descifrando solo el resultado agregado.

El mecanismo distintivo es *cast-or-audit*: tras preparar la papeleta cifrada, el votante elige entre emitirla o auditarla. Si audita, el sistema revela la aleatoriedad r y el votante verifica que el cifrado correspondía a su intención; esa papeleta se descarta y repite el proceso. Un servidor deshonesto que cifrara mal las papeletas sería detectado con probabilidad creciente, sin que ninguna papeleta emitida quede comprometida. Helios se ha usado en elecciones de la ACM/IACR y en varias universidades.

5.2 Scantegrity II

Referencia: Chaum *et al.* (2008) [?].

Diseñado para papeletas de lectura óptica en papel. Cada opción de cada papeleta lleva un código de confirmación impreso con tinta invisible que solo aparece al marcar. La autoridad publica *antes* de la elección compromisos a la tabla completa de códigos; después publica las aperturas parciales necesarias para el recuento.

El votante anota su código y verifica en línea que aparece publicado. El código no revela la opción a nadie más, de modo que no sirve como recibo vendible.

Uso vinculante en una elección pública

Scantegrity II se empleó en las elecciones municipales de Takoma Park, Maryland, en 2009 y 2011. Es, hasta donde alcanza la literatura, el primer uso de un sistema de votación verificable de extremo a extremo en una elección gubernamental vinculante en Estados Unidos [?].

5.3 Otros sistemas desplegados

Sistema	Mecanismo	Referencia
Prêt à Voter	Papeleta con orden permutado comprometido	[?]
Belenios	ElGamal + pruebas de Fiat–Shamir	[?]
ElectionGuard	ElGamal exponencial + Chaum–Pedersen	[?]
IVXV (Estonia)	Mixnet verificable con compromisos	[?]

6 Claves umbral y custodia distribuida

6.1 El artículo original de Pedersen

Conviene recordar que [?] no trata sobre dinero: su título es «compartición verificable de secretos no interactiva e incondicionalmente segura». El problema es repartir un secreto s entre n partes de modo que t de ellas puedan reconstruirlo, y que cada parte pueda *verificar* que recibió un fragmento legítimo sin confiar en el repartidor.

El repartidor elige polinomios f, f' de grado $t - 1$ con $f(0) = s$, publica los compromisos $C_k = g^{a_k} h^{b_k}$ de sus coeficientes, y entrega a cada parte $(s_i, t_i) = (f(i), f'(i))$. La verificación es una identidad de grupo:

$$g^{s_i} h^{t_i} \stackrel{?}{=} \prod_{k=0}^{t-1} C_k^{i^k}.$$

6.2 Generación distribuida de claves

Referencia: Gennaro *et al.* (2007) [?].

En DKG no hay repartidor de confianza: cada participante actúa como repartidor de un secreto propio y la clave final es la suma. Gennaro, Jarecki, Krawczyk y Rabin demostraron que el protocolo ingenuo basado en Feldman-VSS permite a un adversario sesgar la distribución de la clave pública resultante, y dieron una corrección basada precisamente en compromisos de Pedersen —cuyo ocultamiento perfecto impide ese sesgo.

6.3 Firmas umbral en producción

Protocolo	Firma	Uso desplegado
FROST [?]	Schnorr	Firmas Taproot umbral; validadores distribuidos
GG18 [?]	ECDSA	Custodia institucional de criptoactivos
CGGMP [?]	ECDSA	Custodia con abortos identificables
Pedersen DKG [?]	Genérico	Base de casi todos los anteriores

Estos protocolos sostienen hoy la custodia de activos digitales en instituciones financieras: la clave privada nunca existe completa en ningún dispositivo, y las firmas se producen sin reconstruirla.

7 Credenciales anónimas y privacidad

7.1 Credenciales con revelación selectiva

Referencias: Brands (2000) [?]; Camenisch y Lysyanskaya (2002) [?].

Una credencial anónima permite demostrar atributos —«soy mayor de edad», «resido en este estado»— sin revelar la identidad ni los demás atributos. La construcción típica compromete cada atributo y demuestra en conocimiento cero relaciones sobre los compromisos. Los compromisos en grupos de orden oculto [?] son los que permiten pruebas de rango eficientes sobre enteros, necesarias para las comparaciones de edad o de ingreso.

Implementaciones: Idemix de IBM (basado en Camenisch–Lysyanskaya), U-Prove de Microsoft (basado en Brands), y las firmas BBS+ [?], que son la base criptográfica del estándar de credenciales verificables del W3C y de varios pilotos de identidad digital gubernamental.

7.2 Privacy Pass y tokens anónimos

Referencia: Davidson *et al.* (2018) [?].

Resuelve un problema cotidiano: los usuarios de Tor y VPN enfrentan CAPTCHAs repetidos. Privacy Pass permite canjear un CAPTCHA resuelto por varios tokens anónimos, no vinculables entre sí ni con el canje original. Cloudflare lo desplegó en 2017; el mecanismo se estandarizó posteriormente en el IETF y Apple lo adoptó como base de *Private Access Tokens* en iOS 16.

8 Subastas, aleatoriedad y cómputo seguro

8.1 Subastas de sobre cerrado

Referencia: Bogetoft *et al.* (2009) [?].

El caso documentado más citado de cómputo multiparte en producción real es la subasta de contratos de remolacha azucarera en Dinamarca (enero de 2008). Unos 1200 agricultores debían negociar precios con Danisco, el único procesador del país; ninguna de las partes aceptaba revelar sus curvas de oferta y demanda a la otra ni a un tercero.

La solución fue una subasta de doble sentido ejecutada mediante cómputo multiparte entre tres servidores (Danisco, la asociación de agricultores y la universidad), con compromisos sobre las ofertas individuales. El precio de equilibrio se calculó sin que ninguna oferta individual fuera revelada jamás. El sistema procesó ofertas por decenas de millones de euros.

Por qué este caso importa

Es la refutación empírica del argumento de que el cómputo seguro es un ejercicio teórico. El sistema se usó porque *no existía alternativa aceptable*: ninguna de las partes confiaba en la otra, y contratar un tercero neutral resultaba políticamente inviable. La criptografía no compitió contra una solución más barata; compitió contra un estancamiento.

8.2 Aleatoriedad pública verificable

Referencias: Syta *et al.* (2017) [?]; Boneh *et al.* (2018) [?].

Muchos sistemas necesitan aleatoriedad que nadie pueda manipular: loterías, selección de comités, asignación de auditorías. El patrón *commit-reveal* es la construcción natural —cada participante se compromete a un valor, luego todos revelan, y el resultado es el XOR o la suma— pero tiene un defecto estructural.

El ataque del último en revelar

En un protocolo *commit-reveal*, el último participante en revelar conoce ya todas las demás aportaciones y puede calcular el resultado antes de decidir. Su única opción de manipulación es *abstenerse* de revelar, lo que aborta la ronda o altera el resultado. Con un bit de influencia por ronda y participación en múltiples rondas, el sesgo acumulable es significativo. El RANDAO de Ethereum sufre exactamente esta limitación. Las soluciones propuestas son las funciones de retardo verificable (VDF) [?], que imponen un tiempo de cómputo secuencial irreducible entre la revelación y el resultado, eliminando la ventaja informativa del último participante.

El servicio drand, operado por la League of Entropy, produce aleatoriedad pública mediante firmas umbral BLS en lugar de *commit-reveal*, evitando el problema por construcción. Es usado por Filecoin y por el mecanismo de *timelock encryption* de flock.

8.3 Almacenamiento verificable

Referencias: Juels y Kaliski (2007) [?]; Shacham y Waters (2008) [?]; Fisch (2019) [?].

Las pruebas de recuperabilidad permiten a un cliente verificar que un servidor sigue almacenando su archivo sin descargarlo, mediante compromisos a bloques codificados y auditorías aleatorias. Filecoin lleva esto a producción con pruebas de replicación y de espacio-tiempo, donde los mineros demuestran periódicamente que conservan copias físicamente distintas de los datos comprometidos.

9 Aprendizaje automático auditable

9.1 Inferencia verificable

Referencia: Kang *et al.* (2022) [?].

El problema: un proveedor afirma que una respuesta proviene del modelo M que declaró usar, no de uno más barato. La construcción publica un compromiso a los pesos del modelo y acompaña cada inferencia con una prueba de conocimiento cero de que la salida corresponde a la ejecución de esos pesos comprometidos sobre la entrada dada.

El costo sigue siendo considerable —órdenes de magnitud sobre la inferencia directa— y las demostraciones publicadas se limitan a redes de tamaño modesto, pero la línea es activa y directamente relevante para la auditoría de sistemas desplegados.

9.2 Compromiso a datos de entrenamiento y evaluación

Un uso metodológicamente más inmediato, y que no requiere pruebas costosas, es el compromiso a conjuntos de evaluación. Publicar $H(\text{conjunto de prueba} \| r)$ antes de entrenar permite demostrar después que el conjunto no se ajustó a los resultados. Es la traslación directa de la preinscripción de hipótesis al contexto del aprendizaje automático, y aborda de manera verificable el problema de la contaminación de *benchmarks*.

Una aplicación natural para un laboratorio académico

El costo de implementar esto es esencialmente nulo —un hash con nonce, publicado con sello de tiempo— y la ganancia metodológica es sustancial: convierte una afirmación de integridad experimental en una verificable. Para estudios comparativos entre modelos locales y APIs frontera, comprometerse públicamente al banco de problemas y a la rúbrica de evaluación antes de ejecutar los experimentos elimina de raíz la sospecha de selección posterior de métricas.

10 Fallos documentados

Los errores reales son instructivos porque casi nunca están en la matemática del esquema, sino en su integración.

Caso	Falla	Lección
CVE-2012-2459 (Bitcoin)	Duplicar transacciones producía la misma raíz de Merkle con un árbol distinto	La codificación debe ser inyectiva; el árbol debe distinguir hojas de nodos internos
Ataque de segunda preimagen en Merkle	Un nodo interno puede pasar por hoja si no se etiquetan	Prefijar dominios: $H(0x00\ \cdot)$ para hojas, $H(0x01\ \cdot)$ para nodos
Subastas en contratos inteligentes	$c = H(\text{oferta})$ sin nonce, con ofertas redondeadas	Fuerza bruta trivial sobre espacios pequeños; el nonce no es opcional
RANDAO (sesgo del último)	El último en revelar puede abstenerse	Commit-reveal no basta para aleatoriedad pública; se requiere VDF o firmas umbral
Reutilización de aleatoriedad	$\text{Com}(m_1; r) / \text{Com}(m_2; r) = g^{m_1 - m_2}$	Nunca reutilizar r : revela la diferencia de mensajes
Generación indebida de h	$h = g^s$ con s conocido por el implementador	h debe derivarse por <i>hash-to-curve</i> verificable públicamente
Compromisos sin ligadura al contexto	Un compromiso válido se reproduce en otra sesión	Incluir identificadores de sesión y de ronda en el mensaje comprometido

Patrón recurrente

Seis de los siete casos anteriores comparten una estructura: el esquema de compromiso era correcto y la implementación también, pero *el objeto comprometido no era el que el protocolo necesitaba*. Faltaba un nonce, una etiqueta de dominio, un identificador de sesión o una restricción de rango. La verificación formal del esquema aislado no captura estos errores; solo el análisis del protocolo completo lo hace.

11 Tabla de referencia general

Sistema	Esquema	Propósito	Referencia
Dinero confidencial			
Liquid / Elements	Pedersen	Montos y tipos de activo ocultos	[?, ?]
Monero RingCT	Pedersen + Bulletproofs	Montos ocultos con conservación verificable	[?, ?]
Grin / Beam	Pedersen	Poda de la cadena por homomorfía	[?, ?]
Zcash Sapling	Hash de Pedersen	Compromisos de nota en circuito	[?]
Zcash Orchard	Sinsemilla	Compromisos optimizados para Halo 2	[?]
Escalabilidad			
Ethereum EIP-4844	KZG	Compromisos a blobs de datos	[?]
Celestia	KZG + Reed–Solomon	Muestreo de disponibilidad	[?]
StarkNet	FRI	Pruebas transparentes de ejecución	[?]
Verkle (propuesta)	KZG vectorial	Testigos de estado compactos	[?]
Transparencia			
Certificate Transparency	Merkle	Registro auditable de certificados	[?, ?]
CONIKS / SEEMless	Merkle con privacidad	Transparencia de claves	[?, ?]
Parakeet (WhatsApp)	Directorio auditable	Transparencia a escala de producción	[?]
Sigstore / Rekor	Merkle	Procedencia de artefactos de software	[?]
Bitcoin	Merkle	Raíz de bloque; pruebas SPV	[?]
Votación			
Helios	ElGamal exponencial	Recuento homomórfico auditable	[?]
Scantegrity II	Hash	Códigos de confirmación comprometidos	[?]
Prêt à Voter	Permutación comprometida	Papeleta con orden aleatorio	[?]
Belenios	ElGamal	Verificabilidad universal	[?]
ElectionGuard	ElGamal + Chaum–Pedersen	Kit verificable de código abierto	[?]
Claves umbral			
Pedersen VSS	Pedersen	Compartición verificable	[?]
DKG (GJKR)	Pedersen	Generación sin repartidor confiable	[?]
FROST	Pedersen	Firmas Schnorr umbral	[?]
GG18 / CGGMP	Pedersen + Paillier	Custodia ECDSA umbral	[?, ?]
Privacidad y otros			
Idemix / U-Prove	Fujisaki–Okamoto	Credenciales con revelación selectiva	[?, ?]
BBS+ / W3C VC	Pedersen sobre pares	Credenciales verificables estándar	[?]
Privacy Pass	Tokens ciegos	CAPTCHAs anónimos no vinculables	[?]

Sistema	Esquema	Propósito	Referencia
Subasta danesa	Compromisos MPC	Precio de equilibrio sin revelar ofertas	[?]
drand	Firmas umbral BLS	Aleatoriedad pública sin sesgo	[?]
Filecoin	Merkle + PoRep	Almacenamiento verificable	[?]
zkML (EZKL)	KZG	Inferencia verificable de modelos	[?]

Referencias

- [1] B. Adida. *Helios: Web-based Open-Audit Voting*. USENIX Security Symposium 2008, pp. 335–348.
- [2] M. Al-Bassam, A. Sonnino, V. Buterin, I. Khoffi. *Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients*. Financial Cryptography 2021, LNCS 12675, pp. 279–298.
- [3] M. H. Au, W. Susilo, Y. Mu. *Constant-Size Dynamic k -TAA*. SCN 2006, LNCS 4116, pp. 111–125. (Origen de las firmas BBS+.)
- [4] J. Benaloh, M. Naehrig, O. Pereira, D. S. Wallach. *ElectionGuard: a Cryptographic Toolkit to Enable Verifiable Elections*. USENIX Security Symposium 2024.
- [5] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, M. Virza. *Zerocash: Decentralized Anonymous Payments from Bitcoin*. IEEE Symposium on Security and Privacy 2014, pp. 459–474.
- [6] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. *Fast Reed–Solomon Interactive Oracle Proofs of Proximity*. ICALP 2018, LIPIcs 107, art. 14.
- [7] P. Bogetoft, D. L. Christensen, I. Damgård, M. Geisler, T. Jakobsen, M. Krøigaard, J. D. Nielsen, J. B. Nielsen, K. Nielsen, J. Pagter, M. Schwartzbach, T. Toft. *Secure Multiparty Computation Goes Live*. Financial Cryptography 2009, LNCS 5628, pp. 325–343.
- [8] D. Boneh, J. Bonneau, B. Bünz, B. Fisch. *Verifiable Delay Functions*. CRYPTO 2018, LNCS 10991, pp. 757–788.
- [9] S. A. Brands. *Rethinking Public Key Infrastructures and Digital Certificates: Building in Privacy*. MIT Press, 2000.
- [10] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, G. Maxwell. *Bulletproofs: Short Proofs for Confidential Transactions and More*. IEEE Symposium on Security and Privacy 2018, pp. 315–334.
- [11] V. Buterin. *Verkle Trees*. Nota técnica, 2021. <https://vitalik.eth.limo/general/2021/06/18/verkle.html>
- [12] J. Camenisch, A. Lysyanskaya. *A Signature Scheme with Efficient Protocols*. SCN 2002, LNCS 2576, pp. 268–289.
- [13] R. Canetti, R. Gennaro, S. Goldfeder, N. Makriyannis, U. Peled. *UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts*. ACM CCS 2020, pp. 1769–1787.
- [14] M. Chase, A. Deshpande, E. Ghosh, H. Malvai. *SEEMless: Secure End-to-End Encrypted Messaging with Less Trust*. ACM CCS 2019, pp. 1639–1656.
- [15] D. Chaum, R. Carback, J. Clark, A. Essex, S. Popoveniuc, R. L. Rivest, P. Y. A. Ryan, E. Shen, A. T. Sherman. *Scanegrity II: End-to-End Verifiability for Optical Scan Election Systems Using Invisible Ink Confirmation Codes*. EVT 2008.
- [16] V. Cortier, P. Gaudry, S. Glondu. *Belenios: A Simple Private and Verifiable Electronic Voting System*. En *Foundations of Security, Protocols, and Equational Reasoning*, LNCS 11565, 2019, pp. 214–238.
- [17] I. Damgård, E. Fujisaki. *A Statistically-Hiding Integer Commitment Scheme Based on Groups with Hidden Order*. ASIACRYPT 2002, LNCS 2501, pp. 125–142.
- [18] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, F. Valsorda. *Privacy Pass: Bypassing Internet Challenges Anonymously*. Proceedings on Privacy Enhancing Technologies 2018(3), pp. 164–180.
- [19] V. Buterin, D. Feist, D. Loerakker, G. Kadianakis, M. Garnett, M. Taiwo, A. Dietrichs. *EIP-4844: Shard Blob Transactions*. Ethereum Improvement Proposals, 2022. <https://eips.ethereum.org/EIPS/eip-4844>
- [20] B. Fisch. *Tight Proofs of Space and Replication*. EUROCRYPT 2019, LNCS 11477, pp. 324–348.
- [21] R. Gennaro, S. Jarecki, H. Krawczyk, T. Rabin. *Secure Distributed Key Generation for Discrete-Log Based Cryptosystems*. Journal of Cryptology 20(1), 2007, pp. 51–83.
- [22] R. Gennaro, S. Goldfeder. *Fast Multiparty Threshold ECDSA with Fast Trustless Setup*. ACM CCS 2018, pp. 1179–1194.

- [23] S. Heiberg, T. Martens, P. Vinkel, J. Willemson. *Improving the Verifiability of the Estonian Internet Voting Scheme*. E-Vote-ID 2016, LNCS 10141, pp. 92–107.
- [24] T. E. Jedusor. *Mimblewimble*. Documento anónimo, julio 2016.
- [25] A. Juels, B. S. Kaliski. *PORs: Proofs of Retrievability for Large Files*. ACM CCS 2007, pp. 584–597.
- [26] D. Kang, T. Hashimoto, I. Stoica, Y. Sun. *Scaling up Trustless DNN Inference with Zero-Knowledge Proofs*. arXiv:2210.08674, 2022.
- [27] C. Komlo, I. Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. SAC 2020, LNCS 12804, pp. 34–65.
- [28] J. Kuszmaul. *Verkle Trees*. MIT PRIMES, 2019.
- [29] H. Malvai, L. Kokoris-Kogias, A. Sonnino, E. Ghosh, E. Oztürk, K. Lewi, S. Lawlor. *Parakeet: Practical Key Transparency for End-to-End Encrypted Messaging*. NDSS 2023.
- [30] G. Maxwell. *Confidential Transactions*. Nota técnica, 2015.
- [31] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, M. J. Freedman. *CONIKS: Bringing Key Transparency to End Users*. USENIX Security Symposium 2015, pp. 383–398.
- [32] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.
- [33] Z. Newman, J. S. Meyers, S. Torres-Arias. *Sigstore: Software Signing for Everybody*. ACM CCS 2022, pp. 2353–2367.
- [34] S. Noether, A. Mackenzie, Monero Research Lab. *Ring Confidential Transactions*. Ledger 1, 2016, pp. 1–18. (ePrint 2015/1098.)
- [35] T. P. Pedersen. *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*. CRYPTO 1991, LNCS 576, pp. 129–140.
- [36] A. Poelstra. *Mimblewimble*. Nota técnica, octubre 2016.
- [37] A. Poelstra, A. Back, M. Friedenbach, G. Maxwell, P. Wuille. *Confidential Assets*. Financial Cryptography Workshops 2018, LNCS 10958, pp. 43–63.
- [38] B. Laurie, A. Langley, E. Kasper. *Certificate Transparency*. RFC 6962, IETF, 2013.
- [39] B. Laurie, E. Messeri, R. Stradling. *Certificate Transparency Version 2.0*. RFC 9162, IETF, 2021.
- [40] P. Y. A. Ryan, D. Bismark, J. Heather, S. Schneider, Z. Xia. *Prêt à Voter: A Voter-Verifiable Voting System*. IEEE Transactions on Information Forensics and Security 4(4), 2009, pp. 662–673.
- [41] H. Shacham, B. Waters. *Compact Proofs of Retrievability*. ASIACRYPT 2008, LNCS 5350, pp. 90–107.
- [42] E. Syta, P. Jovanovic, E. K. Kogias, N. Gailly, L. Gasser, I. Khoffi, M. J. Fischer, B. Ford. *Scalable Bias-Resistant Distributed Randomness*. IEEE Symposium on Security and Privacy 2017, pp. 444–460.
- [43] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox. *Zcash Protocol Specification*. Electric Coin Company, versión vigente.